

一、 现有问题

目前，数据仓库系统在使用过程中，发现许多设计不足问题。

1. 数据建模

现有数仓模型建设未采用规范化的建模方法论。
导致数仓分层不明确，维度模型混乱，各个业务线的数据孤立。
没有建立统一的维度表，事实表，以及指标。
并且缺乏对应的落地文档，导致维护，迭代困难。

2. 规范

逻辑设计阶段，表名称，字段名称未按照规范命名。
导致表名混乱，未能通过表名来了解表的大体用途。
部分字段命名中，采用了关键字，并且针对同一维度、事实、维度命名存在不一致的问题。

物理设计阶段，针对部分表未考虑其生命周期。
分区字段未适当考虑其二级分区。
缺乏对应的注释。

ETL 过程中，SQL 的编写，未按照指定的规则。
SELECT，WHERE 等关键字大小写混合使用。
子查询嵌套超过 3 层，阅读困难。
对于 WITH AS 子句，别名随意。
并且缺乏对应关键性注释。

3. 数据安全以及质量

目前缺乏对应的数据质量设置。
针对核心字段未设置对应的验证规则。

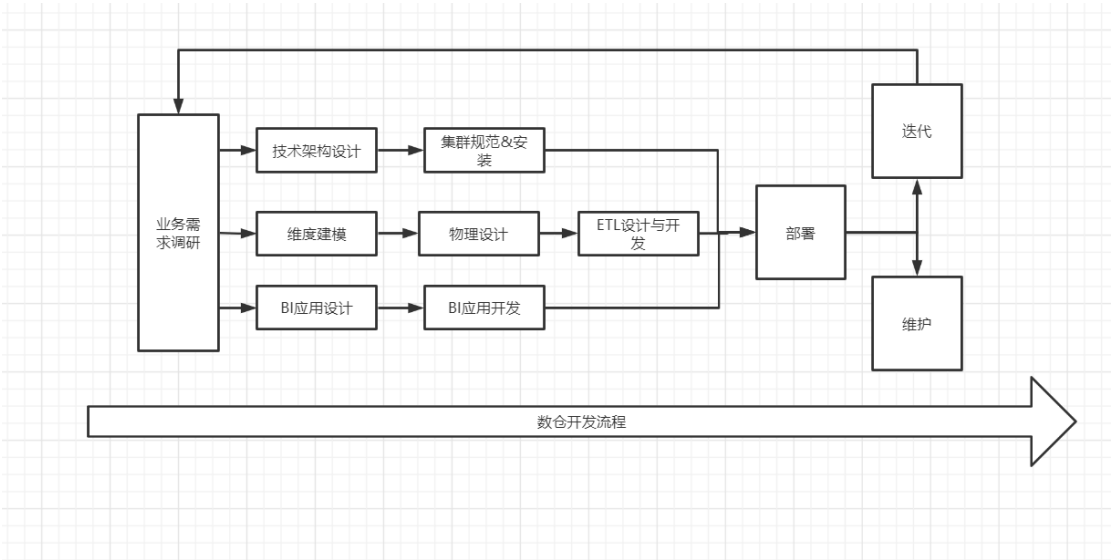
二、 愿景

希望通过这次对数仓的优化，建立起相对规范的流程，设计，并落地对应文档。
数仓体系：高内聚，低耦合，避免重复计算；高性能，高扩展性，适应快速迭代。
数据质量：需求指标治理；数据质量监控。
数据安全：数据脱敏；层级、表、字段级别安全验证。

三、 实施步骤

本章按照数仓开发流程，来制定的实施步骤。

1. 整体开发流程



数据仓库开发流程如上，包含了数仓建设的整个生命周期。
由于采用阿里云 DataWorks 大数据开发组件，因此无需过多关注架构设计、集群安装、BI 应用设计和应用步骤。
核心流程包含：需求分析、维度建模、物理设计、ETL 设计与开发。

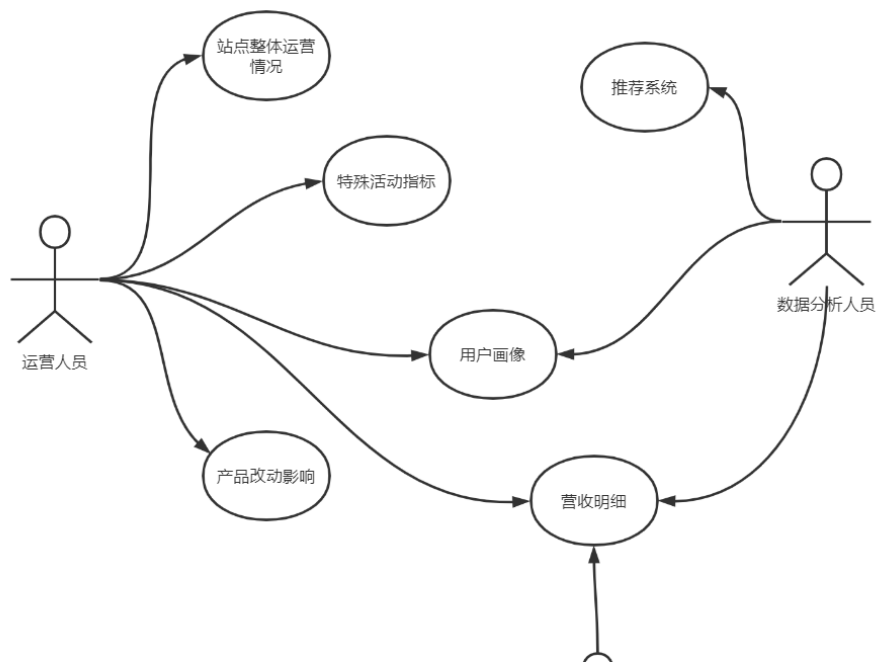
2. 业务需求分析

此阶段产物：需求文档

A. 需求用例

每个用例提供了一个或多个场景，该场景说明了系统是如何和最终用户或其它系统互动，也就是谁可以用系统做什么，从而获得一个明确的业务目标。

示例：电商站点的需求用例



B. 需求列表

需求列表直接与指标治理有直接的联系,并且指导后续的业务过程分析,是非常重要的步骤。需求描述文档,包含各个指标的名称、计算口径、样例代码、负责人、变更历史、输出历史等信息。

业务场景: 用例中的业务需求

指标名称/简写: 参考下节对于指标命名规则。

指标定义: 详细描述指标

输出方式: 接口, 报表, Excel.

输出周期: 实时, 周, 天, 月

过滤条件: 描述核心过滤数据

维度: 时间, 用户等维度

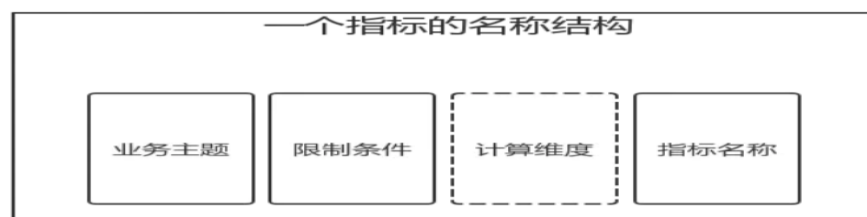
示例: 指标明细定义

1	A	B	C	D	E		G		I	J
	业务场景	指标名称	简写	指标定义	过滤条件	维度	输出方式	输出周期		
3		上线商品	ubm_prd	按商品关联的框架协议的采购类型统计,		天, 月	报表	天		
4		上线数量	ubm_prd_num	按采购类型 = 直采/组采/自采, 协议甲方=企业, 框架下有上架商品 统计协议数量				天		
5	框架协议统计	上线率	ubm_prd_rt	上线框架数量/总框架数				天		
6										
7										
8										

度量术语表:

度量	全称	简写	说明
总个数	count	cnt	
总金额	sum	um	
比率	ratio	rt	
留存率	Retention_rate	ret_rt	前3个字符+组合rt简写

C. 指标命名规则



每个数据指标，其实都可以分解为“业务主题+限制条件+计算维度+指标名称”的结构。

业务主题，可理解为指标应用的业务范围。

限制条件，即是指标计算时需要处理的某些必要条件（不做处理指标会脱离业务意义）。比如，网站注册转化率计算时剔除爬虫访问；

计算维度，即指标做某种维度切分后进行的汇总。比如城市、时段、区域等。

最后是指标名称，要求名副其实且简洁易懂。

将各个结构进行罗列并用“-”连接后，就是一个清晰的指标名称。

每部分结构中，有时会同时出现多个条件，若是交集，建议用“-”符号连接，若是并集，用“&”符号连接。

每部分结构中，有时会同时出现多个条件，若是交集，建议用“-”符号连接，若是并集，用“&”符号连接。除了业务主题外，其他结构都是可以设置默认值，进而在名称中隐去，使得指标名称形成“主题-名称”的简写形式。

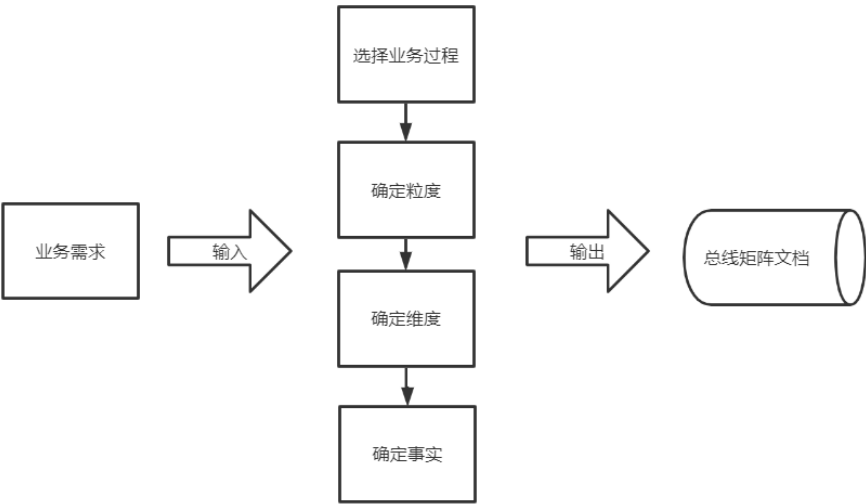
当某些场景下（数据产品开发、可视化呈现等），业务主题加名称的形式已然过长，可只保留名称，而将业务主题变成注释形式另外展示。

示例：

全称，博客-登录-发博-全维度-留存率；
简称，博客-发博-留存率。

3. 维度建模

维度建模是数仓建设的核心，包含选择业务过程，声明粒度，确定维度，确定事实 4 个步骤。
采用总线矩阵来描述整个业务流程，并落地文档。



此外，公司现包含论坛，博客，学院，下载，问答，VIP 会员等众多业务板块。
确保维度和事实的一致性，整体采用数据仓库总线架构，将各个板块的业务数据汇聚在一起。
避免数据孤岛。

A. 总线矩阵

	业务板块	数据域	业务流程	粒度说明	度量值	公共维度			
						时间	客户端	区域	用户
1	博客	文章	发布/更新文章	每个文章发布1行	√			√	
2			搜索	每个搜索行为1行	√			√	
3			点击	每个点击事件1行	√	√		√	
4			浏览	每个浏览事件1行	√	√		√	
5			更新						
6		分类专栏	删除						
7			新增专栏	每个专栏新增1行					
8			设置为付费专栏						
9			删除						
10			启动						
11	用户	注册							
12		登录							
13		退出							
14	评论								
15									
16									
17	下载								
18									
19									
20									
21									
22									
23									

说明：改图例仅为示例，目的是为了介绍总线矩阵。

总线矩阵识别的是业务过程，每行代表组织的业务过程，每列代表公共维度，
通过打标示的方式来标示事实与维度之间存在关联。
总线矩阵是数仓建设中最重要产物之一，用于结构规划、数据库设计、数据治理的基础。

此外，由于在详细建模过程中，或者新增需求，都有可能引入新的事实表和维度表以支持新的业务流程，或者是对维度表重新进行划分，因此要始终更新或增量式维护总线矩阵。

B. 维度模式

维度表的处理占数仓建模的 80%以上，是总线架构的核心。需要在充分了解各个业务板块、数据的前提下，制定一致性的维度。

常用的维度模型包含：雪花模型，星型模型，星系模型。通常情况下，混合式各种模型，来搭建数仓，但是我们更倾向于采用星型模型，因为不同于 OLTP 系统的 3TNF 范式，OLAP 系统更注重以冗余数据加快多维分析的查询性能，以及灵活性。

以下列出了常见的维度处理方法，在合适的场景应用不同的模式。

a) 代理键

一般情况下，是不会使用代理键，因为代理键增加了 ETL 处理的复杂性。
但是当需要对多源同一维度的不同维表进行整合时，是需要使用代理键，解决自然键不唯一的问题。
代理键的生成可以使用 Hive 中自带的窗口函数 `row_number()` 来实现。

b) 多层次级维度

同一个维度表中存在不同的层级。
通常像，商品类别、目录，区域都是层次维度。
需要进行非规范化的扁平处理。

示例：
ods_blog_blog_channel_[base|log]（博客分类表）

字段名	数据类型	说明
Id	Bigint	主键
Name	String	名称
Type	Bigint	类型
parentid	Bigint	父分类 id

逆规范化后：
dwd_blog_blog_channel

字段名	数据类型	说明
Id	Bigint	主键
Name	String	名称
Type	Bigint	类型
Parentid	Bigint	父分类 id

ParentName	String	父分类名称
ParentType	Bigint	父分类类型

c) 维度退化

对于简单的维度来说，不需要创建对应的维度表。
直接在事实表中保存对应的编码，或者值。

d) 杂性维度

可以将多个简单维度，进行 Cross join，组成杂性维度。

e) 时间维度

构建同一的时间维度。

示例：dim_pub_date（公共时间维度）

说明：该表记录年，季度，月，旬，日的日期信息，跟进 DT_TYPE 把各种粒度的时间分开，多种日期粒度信息合并放在该表中。

数据来源：程序生成

属性	字段	数据类型	说明
Date_SK	代理键	Int(10)	D:YYYYMMDD 格式，M: YYYYMM
DT_TYPE	时间类型	String	D:天；X:旬； M:月；Q: 季度；Y: 年
YEAR_NM	年名称	String	
YEAR_CODE	年编码	String	YEAR_CODE
HALF_YEAR_CODE	半年编码	String	
HALF_YEAR_NM	半年名称	String	上半年，下半年
QUARTER_NM	季名称	String	
QUARTER_CODE	季编码	String	编码, 可做排序用
MONTH_NM	月名称	String	
MONTH__CODE	月编码	String	MONTH_CODE
XUN_NM	旬名称	String	
XUN_CODE	旬编码	String	XUN_CODE
WEEK_NM_Y	年度周	String	
WEEK_NM_M	月度周	String	月度周编码(每月 1 日到 7 日为 week1, 依次为 week2, week3, 三周后所有日期为 week4)
FIVE_DAY_CODE	五日编码	String	ETL 产生, 可以作为排序字段用

FIVE_DAY_NAME	五日名称	String	值:第一个五日,第二个五日 规则:, 1-5 日为第一个五日, 6-10 日第二个五日, 一个月最多 6 个, (存在最有一个五日有 7 天或者不够 5 天的情况)
DAY_NM	日名称	String	
DAY_DT	日期	Date	
DAY_DT_N	农历日	Date	对应的农历日期
HOLIDAY	节假日	String	节假日。元旦, 国庆等
W_INSERT_DT	数据插入日期	Date	ETL 插入日期
W_UPDATE_DT	输入更新日期	Date	ETL 更新日期

如果需求需要按照, 月、周、日、节假日、以及周末进行分析。

是需要将时间戳处理, 增加时间维度代理键。

不能再分析的时候, 使用 sql 函数进行统计。

示例: mall_orders_order_base (订单系统订单表)

字段名	类型	说明
Id	Bigint	唯一主键
Order_number	String	订单编号
Pay_time	Bigint	支付时间
Pay_time_id	Int	FK_支付时间维度

增加了 Pay_time_id 字段, 便于基于年, 月, 假期, 周末进行订单的分析。

f) 维度角色扮演

当某个维度在单一事实表中重复出现多次, 则会出现维度角色扮演。基本维度可以作为单一物理表存在, 但是每个不同的角色应该被当做不同的视图。

示例: mall_orders_order_base (订单系统订单表)

字段名	类型	说明
Id	Bigint	唯一主键
Order_number	String	订单编号
Pay_time	Bigint	支付时间
Pay_time_id	Int	FK_支付时间维度
Create_time	Bigint	创建时间
Create_time_id	Int	FK_创建时间维度

增加了 Pay_time_id 和 Create_time_id 字段, 便于基于年, 月, 假期, 周末进行订单的分析。

需要创建 2 个时间视图, 继承自单一的时间维度表。

dim_pub_{pay_date}_view (支付时间视图)
dim_pub_{order_date}_view (下单时间视图)

g) 缓慢变化维度

针对维度表的每一个属性，在源系统数据发生变化时，需要定义对维度表产生的影响。
对于缓慢变化维，不同的数据量，不同的数据需求，处理方式也是不同的，必须采用适当的 SCD 技术。

i. 覆盖

使用新值覆盖旧值，这是最简单的方式，但是无法保留历史值，并且会导致对受影响的属性聚集时产生错误，需要对聚集数据重新进行计算。

ii. 新增行

将新的维度行写入事实表，精确跟踪维度变化历史。
每行新增 start_date, end_date 代表有效时间范围，并且设置 flag 有效标识。
可以查询任何时刻的历史快照。
但是针对数据量较大的维度表，在 join 时会影响性能，此方式只有在特殊情况下才会使用，一般不建议使用。
建议当数据量不大的时候，可以采用每日全量的方式记录历史变化。

iii. 新增列

新增属性，代表上一次的属性值。能够确保要么通过新的，要么通过旧的属性值来统计新的和历史事实表。

iv. 其他方式

除了以上 3 中方式外，还有微型维度等等扩展方式。
这里就不一一介绍了。

C. 表详细设计

详细的事实表和维度表设计，并落地表设计文档。

事实表设计包含：外键，退化维度，非规范化属性值，度量值（可加，半可加，非可加）

[illegible][illegible]

ETL 设计包含很多方面，比如任务流程、依赖，数据来源概要，表分区，表设计（名称，类型和约束），数据质量跟踪，以及加载频率等。

A. SQL 规范

项	说明	示例
注释	添加必要的注释	
WITH.....AS 字句	使用 with as 字句替换子查询	
简写	遵守术语表的简写规范 同一个含义的维度名称相同	

表别名命名	al_{简写}, 所有字段前都增加别名, 防止字段混乱。	

B. 性能考虑

对数据进行处理前, 要考虑数据量以及预见的行能需求。

当数据量较大时, 要主观性的编写高性能的 SQL, 以及表设置考虑。

常见优化点如下。

- 谓词下推, 将过滤条件写至 on 字句
- 避免不加 limit 限制, 使用 order by 全排序
- 使用 Map join 的方式 join 大-小维度表
- 避免使用 count distinct, 使用 count + group by 子查询替换
- 输入文件避免采用不支持切分的压缩格式
- 避免使用 select * 字句
- 对大-大 join, 先了解数据分布, 再进行处理, 避免数据倾斜问题
- 考虑二级分区, 选择合适的 key 作为二级分区字段

示例:

```
SELECT
    ob.id,
    orb.create_time AS refund_time,
    orb.refund_price,
    orb.refund_cb_price
FROM
    ods_mall_orders_order_base ob
JOIN
    ods_mall_orders_order_refund_base orb
ON
    ob.id = orb.order_id
AND
    orb.refund_status = 3
```

C. 异常数据处理

设置合理的数据质量检测方案。

当数据异常时, 可选择

- ① 中断过程
- ② 将错误记录发送到临时备份表中
- ③ 增加审计字段, 对数据进行标记

一般建议选择第 2 种方式，第一种需全程人工干预；第 3 种会增加审计列；视情况而定。

5. 其他说明

A. 阿里云规范

强烈建议，阅读阿里云 MaxCompute 文档中《规范》章节；里面描述了数据建模规范以及表设计规范，并强制遵守该规则。



B. 数据安全&数据质量

针对核心字段，采用 DataWorks 提供的 数据质量工具 设置校验规则。
针对核心字段，设置对应的安全级别。

C. 术语表

建议建立业务术语表。
